MASTER BUILD PROMPT — DEEPSHIELD AI

Explainable Multimodal Deepfake Detection & AI Security Platform

Act as a senior AI/ML engineer, digital forensics investigator, full-stack architect, cybersecurity engineer, and premium SaaS UI/UX designer.

Build a complete, runnable, security-conscious software platform named **DeepShield AI** that addresses the following problem statement:

Explainable Multimodal Deepfake Detection and AI Security Platform

The application must combine complementary open-source deepfake detection methodologies, forensic analysis, explainable AI, confidence calibration, and evidence fusion. It must help an investigator understand why an image, video, or audio recording may be authentic, manipulated, or AI-generated.

Do not create only a landing page, static prototype, or fake dashboard. Build the actual application, connect the frontend to working backend services, integrate real detection models where technically feasible, and document any unavailable model dependencies honestly.

The project must be suitable for a cybersecurity/AI hackathon demonstration, technical evaluation, and future extension into a research-grade forensic platform.

---

1. PRIMARY OBJECTIVE

Create an integrated platform that accepts uploaded digital media and produces an evidence-based forensic report.

The platform must support:

- Image deepfake detection.
- Video deepfake detection.
- Audio deepfake and voice-spoofing detection.
- Audio-video consistency analysis where compatible inputs and models are available.
- Multiple complementary detection techniques.
- Individual detector scores and evidence.
- An overall, appropriately calibrated deepfake risk score.
- Manipulation localization when supported by an appropriate model.
- Explainable, human-readable forensic findings.
- Reproducible analysis reports and model/version information.
- Secure file handling and model execution.
- A premium, responsive user interface.

Image analysis is the first priority and must be implemented end to end. Video and audio must be developed as modular pipelines with real inference integrations wherever feasible. Never represent an unimplemented detector as operational.

2. TECHNOLOGY STACK

Use the following stack unless a specific component cannot work in the selected deployment environment.

Frontend:

- React.
- TypeScript.
- Vite.
- Tailwind CSS.
- shadcn/ui or an equivalent accessible component library.
- Lucide icons.
- Recharts for charts.
- React Router for navigation.
- React Query or an equivalent for server-state management.

Backend:

- Python 3.11 or another version compatible with selected model dependencies.
- FastAPI.
- Pydantic for validated API schemas.
- Pillow and OpenCV for image/video processing where appropriate.
- PyTorch and Hugging Face tooling only where compatible with the chosen detectors.
- FFmpeg for media decoding where required.
- SQLite for an initial local database, with a clean path to PostgreSQL later.

Testing:

- Pytest for backend tests.
- Vitest and React Testing Library for frontend tests.
- API integration tests.
- Security and file-validation tests.

Architecture:

- Keep frontend and backend separate.
- Use typed API contracts.
- Store model adapters separately from application logic.
- Use environment variables for configuration.
- Provide a complete README, dependency files, environment template, and startup instructions.

Do not install every research repository and its dependencies automatically. Select compatible detectors, isolate conflicting dependencies, and keep the application runnable.

3. PREMIUM UI/UX DESIGN

Create a polished, luxury-grade forensic intelligence dashboard.

Visual direction:

- Dark graphite and near-black backgrounds.
- Restrained electric blue, cyan, and violet accents.
- Clear typography, generous spacing, refined cards, and subtle gradients.
- Professional data visualization and forensic evidence panels.

- Smooth, restrained animations.
- Excellent mobile, tablet, and desktop responsiveness.
- Accessible contrast, keyboard navigation, and clear loading/error states.

Avoid:

- Generic AI landing-page templates.
- Excessive glowing effects.
- Giant unnecessary headings.
- Fake live metrics.
- Decorative charts containing invented data.
- Buttons that do nothing.
- Placeholder analysis results presented as genuine findings.

Build these pages:

A. Dashboard

- Total completed analyses from actual application records.
- Analysis breakdown by media type.
- Recent analyses.
- Real detector health and availability.
- Processing failures and model warnings.
- Risk distribution based only on real stored results.

B. New Analysis

- Drag-and-drop file uploader.
- Browse files button.
- Media type detection.
- Supported file formats and size limits.
- File validation and progress indicator.
- Analysis configuration.
- Privacy notice and processing status.

C. Analysis Results

- Overall risk score.
- Classification and uncertainty.
- Individual detector results.
- Evidence-fusion explanation.
- Heatmaps or suspicious-region overlays when genuinely available.
- Video timeline and selected-frame evidence.
- Audio segment evidence when supported.
- Technical metadata.
- Limitations and alternative explanations.

D. Analysis History

- Search and filters.

- Media type and result status.
- Timestamp and analysis ID.
- Open report, export report, and delete record actions.

E. Model Registry

- Detector name.
- Media modality.
- Research source and repository URL.
- Model version or checkpoint identifier.
- License and usage restrictions.
- Current availability.
- Supported input types.
- Known limitations.
- Whether the detector is installed, operational, or unavailable.

F. Evaluation Center

- Dataset selection.
- Evaluation metrics.
- Confusion matrix.
- ROC-AUC and precision-recall metrics where appropriate.
- Calibration plots.
- Per-dataset and per-manipulation-type performance.
- Reproducible experiment configuration.

G. Settings and Documentation

- Processing configuration.
- File-retention policy.
- Model status.
- Supported formats.
- API health.
- Documentation and methodology.

4. IMAGE ANALYSIS — FIRST PRIORITY

When a user uploads a JPG, JPEG, PNG, or WebP image, execute a real backend analysis pipeline.

Implement the following stages.

Stage 1: Secure ingestion

- Verify actual file content, not just the filename or extension.
- Validate MIME type and image decoding.
- Apply configurable file-size and pixel-count limits.
- Protect against decompression bombs and malformed images.
- Reject unsupported or corrupted files with understandable errors.

- Compute a SHA-256 hash for evidence identification.
- Record dimensions, format, color mode, and available metadata.
- Preserve the original uploaded file unchanged if evidence preservation is enabled.
- Analyze a safe working copy.
- Never execute uploaded content.

Stage 2: Image preprocessing

Implement deterministic preprocessing appropriate to each model.

Include:

- Image decoding.
- RGB conversion where required.
- Model-specific resizing and normalization.
- Handling of very large images.
- Optional tiled or crop-based analysis if supported by the selected model.
- Preprocessing records and model input specifications.

Do not silently apply preprocessing that invalidates a model's expected input.

Stage 3: Complementary detector framework

Build a pluggable detector interface. Each detector should implement a consistent contract containing:

- Detector ID and name.
- Supported modality.
- Model version and checkpoint information.
- Input requirements.
- Inference method.
- Raw outputs.
- Normalized score where justified.
- Evidence and limitations.
- Execution status.
- Timing and error details.

Investigate and integrate compatible image-detection implementations from the repository catalogue below.

Priority candidates:

- UniversalFakeDetect / UnivFD.
- FaceForensics++ / Xception baseline.
- M2TR.
- DCL.
- SBI — Self-Blended Images.
- HiFE.
- M2F2-Det.

- VLAForge.
- SIDA.
- LIDMark.
- DeepfakeBench as a possible unified evaluation and detector-integration framework.

Select at least two complementary image detectors for the initial real multi-detector implementation if their compatible checkpoints, licenses, dependencies, and compute requirements permit it.

Prefer complementary signals, such as spatial image artifacts, frequency-domain characteristics, and semantic/general synthetic-image cues, rather than multiple nearly identical models.

If only one real detector can run in the chosen environment, clearly report that limitation and implement the remaining adapters as unavailable integrations, not as fake models.

Do not assume a repository contains downloadable trained weights. Inspect its documentation and determine whether the checkpoint is accessible and legally usable.

Stage 4: Additional forensic evidence

Implement genuine, clearly labeled forensic checks where useful:

- Image metadata and EXIF inspection.
- File format and encoding information.
- Image dimensions and color characteristics.
- Metadata inconsistencies where demonstrable.
- Optional frequency-domain or residual analysis when technically justified.
- Face detection and face-region analysis only when appropriate and supported.
- Model-based manipulation localization when supported by actual model outputs.

Important:

- Metadata absence is not proof of manipulation.
- Metadata presence is not proof of authenticity.
- Compression artifacts can affect detector performance.
- A generic edge map or noise map must not be labeled as a deepfake localization map.
- If localization is unavailable, explicitly say so.

Stage 5: Evidence fusion

Combine available, valid detector outputs through a configurable fusion component.

Requirements:

- Preserve each detector's individual score.
- Record detector failures.
- Avoid treating missing detectors as zero risk.
- Account for correlated detectors where possible.
- Support validation-based calibration.
- Record the fusion algorithm and its configuration.

- Expose how each detector contributed to the final output.
- Handle disagreement between detectors explicitly.

Initially, use a documented, validation-tuned fusion strategy if suitable calibration data is available. Otherwise, provide an explicitly experimental fusion strategy and label the result uncalibrated.

Never invent weights or claim calibration without supporting validation evidence.

Stage 6: Image result

Display:

- Deepfake risk score from 0 to 100.
- Model confidence or uncertainty as a separate field when available.
- Classification: likely authentic, inconclusive, or likely manipulated.
- Individual detector outputs.
- Number of detectors that succeeded.
- Evidence supporting the assessment.
- Conflicting findings.
- Manipulation heatmap when supported.
- Metadata findings.
- Limitations and model version information.
- Processing time.
- Analysis ID and file hash.

Do not force every result into an absolute fake/real decision. Provide an inconclusive outcome when evidence is insufficient or contradictory.

5. IMPORTANT SCORE DEFINITIONS

Implement these distinctions throughout the UI and backend.

A. Deepfake risk score:

An estimate of how strongly the available evidence supports manipulation or synthetic generation.

B. Model confidence:

A model-specific measure whose meaning depends on the model and output definition.

C. Calibrated probability:

A probability interpretation supported by appropriate held-out validation data and calibration testing.

These are not interchangeable.

Example result formatting:

Deepfake risk: 90/100

Assessment: High estimated manipulation risk

Evidence: Two available detectors indicate synthetic-image patterns

Calibration: Not validated for this input domain

Limitations: Compression and unseen generation methods may affect reliability

This is an illustrative UI example only. Never hardcode it into real analysis responses.

Do not use labels such as “90% certain fake” unless the displayed value has been validated for that interpretation.

Do not assume a model's raw softmax score is a calibrated real-world probability.

6. VIDEO ANALYSIS

Implement a real video analysis pipeline after image inference works.

Support reasonable limits on duration, file size, resolution, and frame count.

Use FFmpeg or OpenCV for safe decoding and frame extraction.

Stages:

- Validate the video.
- Extract basic container and stream metadata.
- Select frames using a deterministic documented sampling strategy.
- Run an appropriate image detector on sampled frames.
- Integrate a genuine temporal detector when its dependencies and weights are available.
- Analyze temporal consistency where supported.
- Aggregate frame-level evidence using a documented method.
- Identify suspicious timestamps or intervals when supported.
- Produce a timeline and forensic report.

Evaluate appropriate research methods from:

- LipForensics.
- RealForensics.
- AltFreezing.
- FTCN.
- TALL / TALL-Swin.
- DeepfakeBench.
- LIDMark.
- FVBench for evaluation of relevant vision-language methods.

Do not treat image-level frame predictions alone as proof of temporal manipulation.

If temporal detectors are unavailable, label the analysis as frame-based and explain its limitations.

Provide:

- Video-level risk score.
- Per-frame results.
- Timeline.
- Selected-frame previews.
- Supported suspicious intervals.

- Detector availability and disagreement.
- Reproducible sampling settings.

7. AUDIO DEEPFAKE AND VOICE-SPOOFING ANALYSIS

Implement a modular audio pipeline using genuine audio models.

Investigate:

- AASIST.
- RawNet2 / RawNet3.
- RawBoost.
- SyncNet for relevant audio-video synchronization analysis.

Use the appropriate repository's documented preprocessing, sample rate, feature extraction, and inference process.

The audio pipeline should:

- Validate uploaded audio.
- Decode supported formats safely.
- Record audio duration and technical metadata.
- Apply required preprocessing.
- Run configured voice-spoofing detectors.
- Produce model-specific scores and supporting evidence.
- Identify relevant suspicious segments only when the model supports segment-level analysis.
- Explain model limitations.

Do not claim to identify a specific speaker's identity or prove that a particular person said something merely because the audio is classified as synthetic.

Do not label every audio recording as deepfake simply because it is noisy, compressed, or unfamiliar.

8. AUDIO-VIDEO CONSISTENCY

When both audio and video are present, allow a separate synchronization analysis.

Investigate SyncNet and LipFD where compatible.

Analyze:

- Audio-video synchronization.
- Lip-motion and speech consistency.
- Temporal evidence where supported.

Report whether a mismatch was detected, how it was measured, and what alternative explanations exist.

A synchronization mismatch alone does not establish a deepfake. Dubbing, editing, latency, and ordinary recording conditions can also cause mismatches.

9. REPOSITORY CATALOGUE — USE THE PROVIDED PDF

Inspect the repositories below and their official documentation before integration.

Image, spatial, frequency, or general synthetic-image candidates:

- FaceForensics++: https://github.com/ondyari/FaceForensics
- Xception / face-forgery baseline: https://github.com/crywang/face-forgery-detection
- UniversalFakeDetect: https://github.com/Yuheng-Li/UniversalFakeDetect
- M2TR: https://github.com/wdrink/M2TR-Multi-modal-Multi-scale-Transformer
- DCL: https://github.com/chenshen03/DCL-DeepFake-Image-Detection
- SBI: https://github.com/mapooon/SelfBlendedimages
- HiFE: https://github.com/Gina-YUE/HiFE
- M2F2-Det: https://github.com/CHELSEA234/M2F2_Det
- VLAForge: https://github.com/mala-lab/VLAForge
- SIDA: https://github.com/hzlsaber/SIDA
- Forensics Adapter: https://github.com/OUC-VAS/ForensicsAdapter
- ProDet: https://github.com/beautyremain/ProDet
- LAA-Net: https://github.com/DepInfoAliciaSHAO/LAA-Net
- DID: https://github.com/Qixinyi1122-lucky/DID
- OSDD: https://github.com/HanHuCAS/OSDD
- RFFR: https://github.com/shiliang26/RFER
- LIDMark: https://github.com/vpsg-research/LIDMark
- DeepfakeBench: https://github.com/SCLBD/DeepfakeBench

Video and temporal candidates:

- LipForensics: https://github.com/ahaliassos/LipForensics
- RealForensics: https://github.com/ahaliassos/RealForensics
- AltFreezing: https://github.com/ZhendongWang6/AltFreezing
- FTCN: https://github.com/yinglinzheng/FTCN
- TALL / TALL-Swin: investigate through the official DeepfakeBench implementation and original paper links.
- FVBench: https://github.com/IntMeGroup/FVBench

Audio and synchronization candidates:

- AASIST: https://github.com/clovaai/aasist
- RawNet: https://github.com/Jungjee/RawNet
- RawBoost: https://github.com/TakHemlata/RawBoost-antispoofing
- SyncNet: https://github.com/joonson/syncnet_python
- LipFD: https://github.com/AaronComo/LipFD

Additional candidates:

- M2F2-Det, VLAForge, and SIDA may provide useful explainability-related research features. Verify their actual outputs before using them.
- DIRE: https://github.com/ZhendongWang6/DIRE
- Self-Blended Images: https://github.com/mapooon/SelfBlendedimages

Some repository names, paths, or implementations in the supplied spreadsheet may be abbreviated, renamed, or unavailable. Verify every repository URL and its current contents. Do not silently substitute an unrelated implementation.

For each selected repository:

- Inspect the official README and license.
- Identify its supported modality and task.
- Determine the required Python and framework versions.
- Identify the exact inference entry point.
- Determine checkpoint availability and download requirements.
- Check dataset restrictions.
- Test inference on a valid sample.
- Record known limitations.
- Document the integration.
- Preserve appropriate attribution and license notices.

Do not claim all listed repositories have been integrated. Include an implementation matrix showing which methods are operational, partially integrated, unavailable, or incompatible.

10. CALIBRATION AND EVALUATION

Build a genuine evaluation framework.

Include:

- Precision.
- Recall.
- F1 score.
- Balanced accuracy.
- ROC-AUC.
- Precision-recall curves.
- Confusion matrix.
- False-positive and false-negative analysis.
- Calibration plots.
- Brier score or another suitable probability-calibration metric.
- Per-dataset and per-manipulation-type metrics where data supports them.

Use appropriate held-out evaluation data.

Prevent data leakage between training and testing splits. Prefer identity-aware or source-aware splits where appropriate for face datasets.

Document:

- Dataset name and provenance.

- License and permitted use.
- Data split.
- Number of samples.
- Class distribution.
- Preprocessing.
- Model checkpoint.
- Software version.
- Random seed.
- Evaluation date.
- Metrics and limitations.

Never invent evaluation results, dataset sizes, benchmarks, or model performance.

If no evaluation dataset is configured, show “Not evaluated” rather than fabricated metrics.

11. FORENSIC EXPLAINABILITY

Every report must distinguish between observed evidence, model inference, and analyst interpretation.

Use structured evidence records with:

- Evidence ID.
- Evidence category.
- Detector source.
- Finding.
- Location or timestamp where supported.
- Score and its definition.
- Confidence or uncertainty if available.
- Explanation.
- Limitations.

Use explainability techniques only when they are appropriate for the selected model.

Possible techniques include:

- Grad-CAM or related attribution methods for compatible architectures.
- Model-provided localization maps.
- Frequency-domain visualizations.
- Frame-level temporal evidence.
- Audio-visual synchronization visualizations.

Do not use a generic visualization as proof of the model's reasoning.

The report should answer:

- What was analyzed?
- Which methods were executed?
- What did each method find?
- Where is the supporting evidence?

- How were the findings combined?
- How reliable is the interpretation?
- What could cause an incorrect result?
- What additional investigation is recommended?

12. FORENSIC REPORT EXPORT

Create a professional downloadable PDF and JSON report.

Include:

- Application and report version.
- Analysis ID.
- File hash.
- File metadata.
- Analysis timestamp.
- Detector names and versions.
- Model checkpoint identifiers.
- Preprocessing details.
- Individual detector results.
- Overall risk score and score definition.
- Fusion methodology.
- Calibration status.
- Supporting evidence.
- Localization outputs when available.
- Warnings and limitations.
- Reproducibility information.

Ensure exported results match the actual backend analysis record.

Use clear wording that the result is an analytical assessment, not definitive proof of authenticity or manipulation.

13. DATABASE AND API DESIGN

Create database models for:

- Analysis.
- File metadata.
- Detector registry.
- Detector execution.
- Evidence item.
- Analysis report.
- Evaluation run.
- Application configuration.

Build APIs such as:

POST /api/v1/analyses

GET /api/v1/analyses/{analysis_id}

GET /api/v1/analyses

DELETE /api/v1/analyses/{analysis_id}

GET /api/v1/models

GET /api/v1/health

GET /api/v1/analyses/{analysis_id}/report

GET /api/v1/evaluations

POST /api/v1/evaluations

Use asynchronous jobs for computationally expensive analysis if required.

Return typed JSON responses with explicit statuses:

- queued
- processing
- completed
- partial
- failed

Include meaningful validation errors and API documentation.

Do not return successful results if the detector failed.

14. SECURITY AND PRIVACY

Treat uploaded media as untrusted forensic evidence.

Implement:

- Strict upload validation.
- File-size and resource limits.
- Safe temporary directories.
- Random server-generated filenames.
- Path traversal prevention.
- No execution of uploaded files.
- No unsafe shell-command construction.
- Secure FFmpeg invocation.
- Restricted processing time.
- Restricted network access for media-processing workers where feasible.
- Safe error messages without stack traces in production.
- Secret management through environment variables.
- Rate limiting or documented local-development alternatives.
- CORS restrictions appropriate to deployment.
- No public exposure of private media files.
- No upload of user media to third-party AI APIs without explicit consent.
- A configurable retention and deletion policy.

- Clear disclosure of what is stored and where.
- Protection against malicious archives and oversized media.

Do not log image contents, audio contents, or sensitive metadata unnecessarily.

For local development, make it possible to run the software without requiring paid APIs.

15. PERFORMANCE AND DEPLOYMENT

Design for free or low-cost development environments, while documenting compute limitations.

- Support CPU inference where compatible.
- Detect unavailable CUDA/GPU acceleration.
- Avoid loading every model into memory at startup.
- Load models on demand or use controlled worker processes.
- Cache model weights appropriately.
- Set timeouts and memory limits.
- Provide actionable dependency errors.
- Avoid downloading enormous datasets or model checkpoints automatically.
- Use environment variables for checkpoint paths.
- Document which features require a GPU or additional storage.

Do not promise that every research model will run on Replit or Lovable's default infrastructure.

If a model cannot execute in the current environment, keep its adapter isolated and document a tested alternative deployment method.

For Lovable, if its environment cannot host the Python inference backend directly, implement the frontend and a clean API integration, and provide the complete separate FastAPI backend rather than replacing actual inference with browser-side fake results.

16. TESTING REQUIREMENTS

Write and execute tests for:

- Valid image uploads.
- Invalid file extensions.
- Invalid MIME types.
- Corrupted images.
- Oversized images.
- Malformed files.
- Missing model weights.
- Detector timeouts.
- Detector exceptions.
- Partial detector availability.
- Conflicting detector outputs.
- Fusion score bounds.
- Calibration status.

- Report consistency.
- File deletion.
- API validation.
- Database persistence.
- Frontend loading and error states.
- Real inference on an appropriate sample when a model and test fixture are available.

Provide synthetic unit-test fixtures for application logic, but never use them to claim real-world detector accuracy.

Include a smoke-test command that checks backend health, model availability, and a configured sample analysis.

17. PROJECT STRUCTURE

Use a maintainable structure similar to:

deepshield-ai/

frontend/

src/

components/

pages/

layouts/

hooks/

api/

types/

charts/

styles/

package.json

backend/

app/

main.py

api/

core/

schemas/

database/

services/

security/

detectors/

base.py

image/

video/

audio/

audiovisual/

fusion/

explainability/

evaluation/

reports/

tests/

requirements.txt

model_configs/

scripts/

sample_data/

docs/

architecture.md

model_registry.md

methodology.md

evaluation.md

security.md

licenses.md

limitations.md

.env.example

.gitignore

README.md

Adjust this structure if the framework requires it, but keep detector logic, inference services, fusion, API routes, and UI components separate.

18. REQUIRED DELIVERABLES

At completion, provide:

- A working frontend.
- A working backend.
- A functioning image-analysis workflow.
- Real model integration for every detector marked operational.
- A model registry and integration status matrix.
- Video and audio pipeline implementations where feasible.
- Explainability and evidence-fusion modules.
- PDF and JSON report export.
- Database persistence and deletion.
- Unit and integration tests.
- A sample environment file.
- A complete README.
- Setup instructions for Windows.
- Setup instructions for Replit or the selected hosting platform.
- A model installation and checkpoint guide.
- A dataset and license compliance guide.

- An architecture diagram.
- An evaluation plan.
- Known limitations and future improvements.
- A demonstration workflow that uses real analysis rather than fabricated results.

19. IMPLEMENTATION WORKFLOW

Work in this order:

Phase 1: Inspect the execution environment and establish the architecture.

Phase 2: Build the premium frontend and working backend API.

Phase 3: Implement secure upload, image preprocessing, file metadata, and SHA-256 hashing.

Phase 4: Verify the repository catalogue and choose compatible image detectors.

Phase 5: Integrate and test the first real detector.

Phase 6: Integrate a second complementary detector if feasible.

Phase 7: Implement evidence fusion, honest risk reporting, and explainability.

Phase 8: Build history, reports, model registry, and evaluation tools.

Phase 9: Add video, audio, and audio-video analysis as separate modules.

Phase 10: Execute tests, resolve errors, and document setup and limitations.

Do not stop after generating the interface. Continue through implementation and testing.

If the environment imposes a limit, finish the highest-priority working vertical slice first, and clearly state what remains.

20. ACCEPTANCE CRITERIA

The project is not complete until the following are verified:

- The frontend launches.
- The backend launches.
- The frontend can communicate with the backend.
- A valid image can be uploaded.
- The image is processed by an actual configured model.
- Detector outputs are returned by the backend.
- The result page displays the actual outputs.
- Risk scoring is documented.
- Calibration status is truthful.
- Detector failures are visible.
- Evidence and limitations are included.
- Reports are generated from actual results.
- History works.

- File deletion works according to the configured retention policy.
- Tests pass or remaining failures are clearly documented.
- Setup instructions are reproducible.
- No fake accuracy metrics or fabricated inference results are present.

At the end, give me a concise implementation summary containing:

- What works now.
- Which models are genuinely integrated.
- Which models need weights or additional infrastructure.
- Which modalities are operational.
- Test results.
- How to run the project.
- How to upload and analyze an image.
- Known limitations.

Final instruction: Prioritize real, reproducible forensic analysis over impressive claims. Build a beautiful, professional application, but never sacrifice scientific honesty, security, or working functionality to make the demonstration appear more capable than it is.</WritingBlock>